PRODUCT REQUIREMENTS DOCUMENT

HelloKitty

Automated Sponsor Outreach; CLI + Web

Version 1.0 · June 2025 · Status: Draft

Phase 1: CLI · Phase 2: Web Wrapper

1. Overview

HelloKitty eliminates the manual first stage of sponsor outreach. It reads a shared Google Sheet of leads, generates personalised cold emails using an LLM, creates Gmail drafts during a team session, and schedules them to send at 9 AM via Google Apps Script; no laptop required after the session ends. Replies are tracked automatically and routed to a human the moment a company responds.

The system is built in two phases. Phase 1 is a CLI tool used internally by the developer. Phase 2 wraps the same logic behind a minimal web UI so the broader FnO team can run it without touching a terminal. The source code stays private throughout; the team only ever sees the web interface.

2. Problem Statement

Each funding round the sponsorship team targets 500+ companies. Today every email is manually drafted and sent from a personal Gmail. This produces:

- 40–60 minutes of copy-paste work per session per person
- Inconsistent email quality across team members
- No centralised tracking; replies get missed, companies get double-emailed
- No protection against a single account over-sending and getting flagged as spam

3. User Personas

3.1 FnO Team Member (Phase 2 primary user)

Technical level: Non-technical. Comfortable with Google Sheets and a web browser. Never touches a terminal.

Current workflow: Manually copy template → swap company name → send from personal Gmail → update sheet manually.

Pain: Repetitive, error-prone, no visibility into what teammates have sent.

Success looks like: Opens the web app, clicks one button, closes laptop. Emails go at 9 AM. Done in under 5 minutes.

3.2 Developer/ Maintainer (Phase 1 primary user)

Technical level: Builds and owns the codebase. The only person with repo access.

Responsibilities: CLI setup, Groq API key, Google Cloud service account, email templates, prompt tuning, deploying Phase 2.

Success looks like: Clean service layer that the web wrapper calls with zero code duplication.

4. Phased Delivery Plan

Phase	Scope & Timeline
Phase 1; CLI	Core engine: sheet reading, LLM drafting, Gmail draft creation, Apps Script scheduling, IMAP reply checking. Used by developer only. Target: 1 week.
Phase 2; Web App	Flask/FastAPI backend exposing Phase 1 services as API routes. Minimal React frontend: Google login → dashboard → one-click batch. Hosted on Railway + Vercel. Team gets URL, not source. Target: 1-1.5 week on top of Phase 1.
Phase 3; Polish	Analytics, reply-rate tracking, per-round reporting. Scoped after Phase 2 is validated in production.

The defining constraint: the service layer (sheets_service.py, draft_service.py, send_service.py, reply_service.py) is written once in Phase 1. The CLI and the web API are both thin wrappers that call the same functions. Nothing gets rewritten in Phase 2.

5. System Architecture

5.1 Repository Structure (dummy)

```
HelloKitty/
├── services/
│   ├── sheets_service.py      # Google Sheets read / write / claim rows
│   ├── draft_service.py       # LLM call → structured email → Gmail draft
│   ├── send_service.py        # Triggered by Apps Script at 9 AM
│   └── reply_service.py       # IMAP inbox scan, reply routing
│
├── cli/
│   └── run.py                 # Phase 1 entry point; calls services
│
├── api/                       # Phase 2; added without touching services/
```

```
│   ├── app.py                  # Flask / FastAPI routes
│   └── auth.py                 # Google OAuth handler
│
├── assets/                     # Attachments (brochure, one-pager)
├── .env.example
├── service_account.json        # Google Sheets API; pre-shipped by dev
└── setup.py                    # Phase 1 only; one-time env + auth setup
```

Final repo structure will be updated on github by tomorrow

5.2 End-to-End Flow

```
┌─ SESSION (laptop on) ──────────────────────────────────────────────┐
│                                                                    │
│  run.py / web UI button pressed                                    │
│    ↓                                                               │
│  sheets_service  →  read rows WHERE status = null                  │
│    ↓  atomically write  status = 'taken by <email>'  (first 50)    │
│    ↓                                                               │
│  draft_service   →  for each row:                                  │
│    ↓  map category → base template                                 │
│    ↓  call Groq API → get {subject, body} JSON                     │
│    ↓  DOUBLE-CHECK: re-read row; still 'taken by me'?              │
│    ↓  yes → create Gmail draft → store draft_id in sheet           │
│    ↓  no  → skip row, log conflict, move to next                   │
│    ↓                                                               │
│  sheets_service  →  mark rows  status = 'queued'                   │
│                                                                    │
│  User closes laptop.                                               │
└────────────────────────────────────────────────────────────────────┘

┌─ 9 AM (Google's servers, no laptop) ──────────────────────────────┐
│                                                                   │
│  Google Apps Script (time trigger, bound to sheet)                │
│    ↓  read rows WHERE status = 'queued'                           │
│    ↓  for each row → GmailApp.getDraft(draft_id).send()           │
│    ↓  write status = 'sent', sent_at = timestamp                  │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘

┌─ DAILY (coordinator runs check_replies.py) ────────────────────────┐
│                                                                    │
│  reply_service   →  IMAP scan, filter auto-replies / OOO           │
│    ↓  genuine reply found → status = 'replied' → alert team        │
│    ↓  no reply after N days → status = 'follow-up due'             │
│      → re-triggers draft_service with follow_up.txt template       │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

5.3 Distributed Sending Model

Every team member uses their own personal Gmail account. Spreading 500 emails across 10 accounts (50 each) makes every account look like a normal human sender. One account blasting 500 emails gets flagged. Ten accounts sending 50 each are invisible to spam filters.

The Google Sheet is the only shared resource. The service account key (service_account.json) is pre-shipped in the repo; no user ever configures sheet access manually.

5.4 Race Condition Handling; Optimistic Locking

Two team members running the bot simultaneously is the core concurrency problem. The solution is a two-stage claim with a double-check commit.

```
  STAGE 1; CLAIM (immediately on run start)
    Read all rows WHERE status = null
    Write status = 'taken by person_a@gmail.com'  ← before any LLM work
    Person B's bot, reading a moment later, sees no null rows → gets different
ones

  STAGE 2; DOUBLE-CHECK (right before creating the Gmail draft)
    Re-read the specific row from the sheet
    If status is still 'taken by person_a@gmail.com'  → create draft, proceed
    If status changed to 'taken by person_b@...'      → skip this row, go to
next

  WHY THIS WORKS
    LLM edition (personalising email for all companies) takes 3–10 seconds per
email.
    It is effectively impossible for two laptops to complete Stage 1
simultaneously,
    race through LLM generation at identical speed, AND hit Stage 2 at the
same moment.
    The double-check is a safety net for an already near-zero edge case.
```

5.5 Status State Machine





6. Data Schema; Google Sheet

The sheet is the single source of truth. Every bot instance reads from and writes to it. No local database. Human team members can open it at any time to see live status.

Column	Type	Required	Description
company_name	String	Yes	Full company name; injected into email greeting and subject
category	Enum	Yes	Tech | FinServ | D2C | Infra | Consulting | Other; maps to base template
contact_email	String	Yes	Outreach email address
contact_name	String	No	First name of POC if known; personalises salutation
context_notes	String	No	Free-text hints for the LLM: past hackathon support, known tech stack, etc.
status	Enum	Yes	null | taken by <email> | queued | sent | replied | follow-up due | follow-up sent | no response
assigned_to	String	Auto	Email of operator who claimed the row; set by bot
draft_id	String	Auto	Gmail draft ID; written by bot after draft creation, used by Apps Script at 9 AM
sent_at	DateTime	Auto	ISO 8601 timestamp written by Apps Script after successful send
last_reply_check	DateTime	Auto	Timestamp of most recent IMAP scan; written by reply_service
notes	String	No	Free-text field for human team members post-reply

7. Functional Requirements

Module 1; Data Ingestion (sheets_service.py)

REQ-1.1 Connect to Google Sheets via the Sheets API using a pre-shipped service_account.json scoped to this sheet only. No user configures sheet access.

REQ-1.2 On each run, read only rows where status = null. Immediately write status = 'taken by <operator_email>' for the first 50 rows before any LLM work begins.

REQ-1.3 Enforce a configurable per-run cap (default 50, set via DAILY_CAP in .env).

REQ-1.4 Skip and log any row missing required fields: company_name, category, contact_email. Never crash on bad data; continue to the next row.

Module 2; AI Drafting (draft_service.py)

REQ-2.1 Call the Groq API (primary) using the model specified in GROQ_MODEL env var. Fall back to a locally running Ollama instance if GROQ_API_KEY is absent; enabling offline development.

REQ-2.2 Map the row's category field to a base template in /templates/<category>.txt. Fall back to default.txt if no match.

REQ-2.3 The prompt must include: the base template, company_name, contact_name (if available), and context_notes. Instruct the model to add one to two specific, natural contextual touches. Explicitly forbid fabricating facts not present in the input.

REQ-2.4 Force structured output: the model must return only a valid JSON object with exactly two keys, subject and body. Retry up to two times on parse failure. If both retries fail, skip the row and log it.

REQ-2.5 Before creating the Gmail draft, re-read the row from the sheet. If status no longer equals 'taken by <operator_email>', skip the row silently and move to the next. This is the double-check commit.

REQ-2.6 Create the Gmail draft via the Gmail API. Attach standard club documentation as configured in ATTACHMENT_1 and ATTACHMENT_2 env vars. Write the returned draft_id to the sheet row.

Module 3; Scheduling(Google Apps Script)

REQ-3.1 A Google Apps Script is bound to the master sheet. The developer sets it up once; it runs on Google's servers permanently. No team member interacts with it.

REQ-3.2 A daily time trigger fires the script at 09:00 IST. The script reads all rows where status = 'queued', calls GmailApp.getDraft(draft_id).send() for each, writes status = 'sent' and sent_at timestamp on success.

REQ-3.3 If a draft_id is missing or the draft has been deleted, the script must log the error to a dedicated 'Bot Errors' sheet tab and skip the row. It must not crash the entire batch.

REQ-3.4 The 9 AM send requires zero team member action after the evening session. Laptops do not need to be on.

Module 4; Reply Tracking (reply_service.py)

REQ-4.1 Connect to the operator's Gmail via IMAP (imap.gmail.com:993, SSL). Scan the inbox for emails whose sender domain matches any domain in the sheet's sent rows.

REQ-4.2 Filter out non-human messages before flagging a reply: check X-Autoreply, Auto-Submitted, and Precedence: bulk headers. Treat messages matching these as non-replies.

REQ-4.3 On genuine reply: set status = 'replied', halt all automated follow-ups for that company permanently, and surface a terminal alert (Phase 1) or web notification (Phase 2).

REQ-4.4 If no reply is detected after FOLLOWUP_DAYS (default 7, configurable), set status = 'follow-up due'. The next bot run picks this up and processes it using follow_up.txt template.

REQ-4.5 reply_service.py is run manually by one designated coordinator once per day. Document this clearly in the README. It is not run by every team member.

Module 5; Phase 1 Setup (setup.py + CLI)

REQ-5.1 setup.py runs all first-time configuration: installs dependencies, validates .env is complete, tests Groq API connectivity, tests Google Sheets API connectivity, tests Gmail OAuth, and confirms the Apps Script trigger is active.

REQ-5.2 Gmail OAuth token is obtained via a single browser popup during setup. The resulting token.json is saved locally and listed in .gitignore. Users never see or touch it again.

REQ-5.3 run.py prints a clean human-readable summary: rows claimed, emails drafted, draft IDs written, any rows skipped and why, and confirmation that 9 AM send is scheduled.

REQ-5.4 All errors visible to the user must be plain English. Stack traces go to error_log.txt only.

Module 6; Phase 2 Web App (api/ + frontend)

REQ-6.1 Users authenticate via Google OAuth. The app requests only the Gmail compose and send scope; not full Gmail read access.

REQ-6.2 The dashboard shows the user's assigned rows with status badges. No configuration UI. No settings page.

REQ-6.3 A single 'Generate & Queue My Batch' button triggers the same draft_service flow as the CLI. A progress indicator shows per-email status as drafts are created.

REQ-6.4 The web app calls the same service functions as the CLI. No business logic lives in api/app.py; it is routing only.

REQ-6.5 OAuth tokens for team members are stored server-side, encrypted at rest. Tokens are scoped to compose/send only and are revocable from the user's Google account settings.

8. Technical Specifications

8.1 Stack

Component	Decision & Rationale
Language	Python 3.10+
LLM; production	Groq API (Llama 3.3 70B, free tier sufficient for 500 emails/round)
LLM; local dev	Ollama (Phi-3.5 Mini or Llama 3.2 3B), fallback when GROQ_API_KEY absent (tbd)
Sheet access	Google Sheets API v4 via service account
Gmail drafts	Gmail API v1 via OAuth 2.0
9 AM scheduler	Google Apps Script (JavaScript, bound to sheet, time trigger)
Reply checking	imaplib (Python stdlib)
Phase 2 backend	FastAPI (lightweight, async, auto-docs)
Phase 2 frontend	React + Tailwind CSS (minimal; login, dashboard, one button)
Phase 2 hosting	tbd

8.2 Environment Variables (dummy)

```
# ── LLM ──────────────────────────────────────────────────────────────
GROQ_API_KEY=
GROQ_MODEL=llama-3.3-70b-versatile
OLLAMA_HOST=
OLLAMA_MODEL=

# ── Gmail ────────────────────────────────────────────────────────────
GMAIL_USER=

# ── Google Sheet ─────────────────────────────────────────────────────
SHEET_ID=
SHEET_TAB=

# ── Behaviour ────────────────────────────────────────────────────────
DAILY_CAP=50
FOLLOWUP_DAYS=7

# ── Attachments ──────────────────────────────────────────────────────
ATTACHMENT_1=
ATTACHMENT_2=
```

8.3 LLM Prompt Constraints

These rules will be hardcoded in draft_service.py and never changed casually:

- System prompt includes 2–3 few-shot examples of good output in the exact JSON format required
- Explicitly state: 'Do not invent facts. Only reference information present in context_notes.'
- End every system prompt with: 'Respond ONLY with a valid JSON object with keys subject and body. No preamble, no markdown, no explanation.'
- Set temperature to 0.7; lower produces generic output, higher destabilises JSON formatting
- If the model fails to return valid JSON twice in a row, skip the row and log it; never send a malformed email

9. User Flows

9.1 Phase 1; First-Time Setup (Developer, once)

1. Clone the repo
2. Copy .env.example to .env; fill in GMAIL_USER and SHEET_ID (two lines)
3. Run: python setup.py
4. A browser window opens → log in with Google → click Allow → window closes automatically
5. Terminal prints: 'Setup complete. Run python run.py to start.'

```
What setup.py does automatically:
  ✓ pip install -r requirements.txt
  ✓ Validates all required .env values are present
  ✓ Tests Groq API key (or warns if missing, confirms Ollama fallback)
  ✓ Tests Google Sheets read access
  ✓ Runs Gmail OAuth → saves token.json
  ✓ Sends a test draft to self → confirms Gmail API is working
  ✓ Checks .gitignore contains token.json and .env
```

9.2 Phase 1; Every Session (Developer, ~5 min active)

6. Run: python run.py
7. Terminal shows: 'Claiming 50 rows... done. Generating drafts... 1/50... 2/50...'
8. On completion: '47 drafts created. 3 rows skipped (see error_log.txt). Emails will send at 9:00 AM IST.'
9. Close laptop. Sleep.

9.3 Phase 2; Team Member Flow (no terminal, ever)

10. Open the URL shared by the developer
11. Click 'Sign in with Google' → one-click auth
12. Dashboard shows their 50 assigned companies with status badges
13. Click 'Generate & Queue My Batch' → watch progress bar
14. Done. Close the browser tab.

10. Security

- .env and token.json are in .gitignore. setup.py verifies this before proceeding and refuses to run if either is staged in git.
- service_account.json is scoped to this specific sheet only, not full Google Drive access. Minimum required IAM role: Sheets Editor.
- Groq API key is used only server-side. Never exposed to the frontend or logged.
- Phase 2 OAuth tokens are stored encrypted server-side. Scope is limited to Gmail compose and send, not inbox read. Users can revoke access at myaccount.google.com/permissions at any time.
- No real lead data (company names, emails) is ever committed to the repo. The sample sheet used for testing uses obviously fake data.
- Logs may contain email addresses. send_log.txt and error_log.txt are gitignored and must not be shared outside the team.

11. Risk Register

Risk	Severity	Mitigation
Two operators claim same row	**High**	Claim written before LLM work (Stage 1). Double-check re-reads row before draft creation (Stage 2). Collision probability after both checks is effectively zero.
Gmail account flagged as spam	**High**	50/day cap per account. Personal accounts with existing send history. Sending happens at 9 AM through Gmail's own servers, not a third-party relay.
Groq API down during session	**Medium**	Bot exits with a clear error before any rows are claimed. No half-claimed rows. Operator retries when Groq recovers, or sets OLLAMA_MODEL for local fallback.
Apps Script fails at 9 AM	**Medium**	Errors written to 'Bot Errors' tab in sheet. Developer is notified. Manual send fallback: operator opens Gmail and sends drafts manually.

SLM hallucinates facts	**Medium**	Prompt explicitly forbids inventing facts. context_notes are operator-supplied. Temperature capped at 0.7.
IMAP flags OOO as genuine reply	**Medium**	Filter on X-Autoreply, Auto-Submitted, Precedence: bulk headers. Test against 20 real OOO samples before first production run.
token.json accidentally committed	**Low**	setup.py checks .gitignore. GitHub secret scanning catches it as secondary net. Token is revocable from Google account settings.
Sheet API quota exceeded	**Low**	At 50 rows × ~4 API calls each = 200 requests per run. Sheets API free quota is 300 requests/minute. No risk at current scale.

12. Success Metrics

Version 1.0 is validated after the first full production round:

- Zero duplicate emails to any single company across all operators in a round
- Zero Gmail accounts flagged or rate-limited
- All 'sent' status updates appear in the sheet within 60 seconds of Apps Script execution
- Reply detection correctly routes at least 90% of genuine human replies, validated against a sample set before first production use
- Operator self-reported session time under 10 minutes, down from 40–60 minutes
- Phase 2 web app: new team member onboarded with zero terminal interaction

13. Out of Scope; Version 1.0

- Automated conversational replies; once a company replies, a human takes over. The bot halts and notifies.
- Lead discovery; finding company names, POC names, and email addresses remains manual. The bot's input is an already-populated sheet.
- Web scraping or automated lead enrichment.
- Per-company custom attachments; all emails in a run get the same standard attachments.
- Multi-language generation.
- Unsubscribe / opt-out tooling.

14. Future Roadmap

Phase 3; Analytics

- Per-round reply rate broken down by category, template, and operator
- A/B template testing with statistical significance tracking
- Email open tracking via a lightweight redirect pixel

Phase 4; Conversational Intelligence

- SLM reads the prior thread and drafts a contextual follow-up; not a generic template
- Human approval step before any AI-drafted reply is sent

Phase 5; Lead Discovery

- Structured input (domain list or LinkedIn URLs) feeds an enrichment pipeline to find POC emails automatically
- Legal and ethical review required before building; data sourcing from public sites has compliance implications